

MASTER BOOK DOCENTE

Manuale Unico Ufficiale di Conduzione, Regia d'Aula e Didattica Applicata (22 Ore)

SCHEDA METADATI DEL MANUALE DOCENTE

Corso: Laboratorio Python + Analisi Dati (22 Ore)

Docente Ufficiale: Arnaldo Morena

Istituzione: ITIS Campobasso

Destinazione d'Uso: Manuale unico di riferimento per la regia, conduzione e disaster recovery in aula.

Struttura del Manuale: 11 Sezioni Complete, Canovaccio 8 Moduli, Cronoprogramma Minuto per Minuto, 7 Scenari di Troubleshooting, 53+ Domande Frequenti con Risposte Risolutive, Benchmark Ufficiali del Project Work ed Appendici Tecniche.

Manuale Unico Ufficiale di Conduzione, Regia d'Aula e Didattica Applicata (22 Ore)

1. EXECUTIVE SUMMARY DEL CORSO

- **Denominazione Ufficiale:** Laboratorio Python + Analisi Dati
- **Docente Responsabile:** Arnaldo Morena
- **Istituzione di Riferimento:** ITIS Campobasso
- **Destinatari:** Studenti tecnici, aspiranti Data Analyst e professionisti junior.
- **Durata Complessiva:** 22 Ore (1.320 minuti netti suddivisi in 8 moduli tematici + Project Work finale).
- **Metodologia Didattica:** **Hands-On Workshop** (40% Spiegazione concettuale e Live Coding guidato, 60% Laboratorio pratico autonomo su casi reali).
- **Caso Aziendale Guida:** *TechStore Italia* - Catena retail di elettronica di consumo con filiali territoriali distribuite.

Obiettivi Formativi Primari

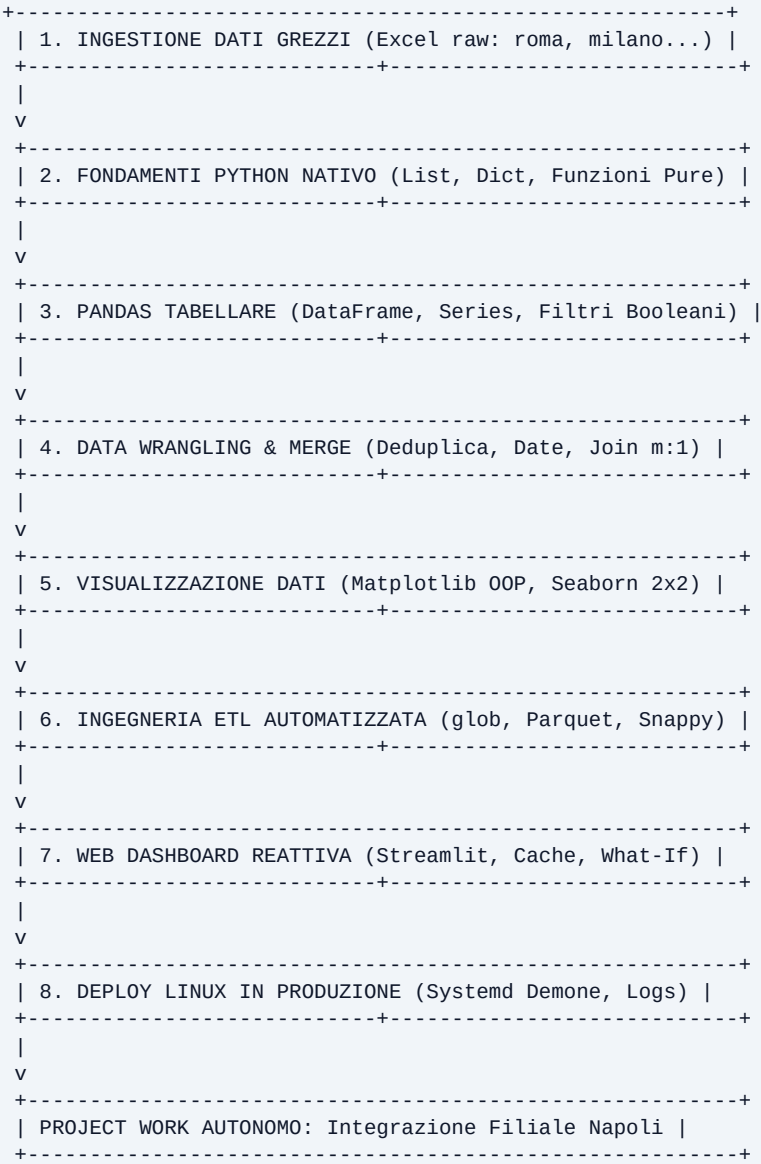
1. **Autonomia Operativa:** Portare i discenti da una conoscenza frammentaria di Excel alla padronanza completa dell'ambiente Python per l'analisi dati.
2. **Ingegneria della Pipeline ETL:** Saper strutturare script batch resilienti capaci di gestire file multipli, bonificare anomalie e memorizzare output ottimizzati su formato Parquet.
3. **Data Visualization Esecutiva:** Saper realizzare visualizzazioni statistiche a livello pubblicazione aziendale (300 DPI, layout 2x2, palette coerenti).
4. **Interactive BI Application:** Costruire web application interattive con Streamlit complete di filtri dinamici e simulatori What-If per il top management.
5. **Produzione & Deploy Linux:** Saper configurare ed orchestrare l'applicazione come servizio di background Linux tramite demone Systemd.

Deliverable Finali Certificati per lo Studente

- **lab01_calcolo_sconti.py:** Script con logica nativa su collezioni **List[Dict]**.
 - **dataset/generated/roma_pulito.xlsx:** Dataset filiale Roma bonificato con merge anagrafico.
 - **dataset/generated/executive_report.png:** Dashboard 2x2 a 300 DPI con formattazione esecutiva.
 - **dataset/generated/dataset_master.parquet:** Master dataset nazionale compresso Snappy.
 - **dataset/generated/report_direzionale.xlsx:** File Excel multi-foglio con aggregazioni pivot.
 - **dashboard/app.py:** Web dashboard Streamlit multi-pagina con reattività immediata.
 - **dashboard_vendite.service:** Unit file Systemd con riavvio automatico e logging **journalctl**.
 - **project_work/dataset_napoli_pulito.parquet:** Integrazione autonoma filiale Napoli (1.000 righe, € 1.042.850,50).
-

2. VISIONE COMPLESSIVA DEL PERCORSO & ARCHITETTURA DIDATTICA

Il percorso è concepito come una transizione fluida e progressiva: dal foglio di calcolo disordinato fino alla moderna piattaforma di business intelligence in cloud/server.



3. AGENDA COMPLETA DELLE 22 ORE

Modulo	Durata	Argomento Didattico	Focus Operativo	Laboratorio Associato
Modulo 1	2h	Python Operativo per l'Analisi Dati	Tipi primitivi, List[Dict] , funzioni pure, calcolo IVA e sconti	laboratori/lab01_python_operativo/
Modulo 2	4h	Pandas Fondamentale	DataFrame, Series, filtri booleani, .loc/.iloc , gestione copie .copy()	laboratori/lab02_pandas_fondamenti/
Modulo 3	3h	Data Wrangling & Qualità del Dato	Deduplicazione, parsing date eterogenee, merge(validate='m:1')	laboratori/lab03_data_wrangling/
Modulo 4	3h	Visualizzazione & Reporting Esecutivo	Matplotlib OOP (fig, ax), Seaborn, palette brand, salvataggio 300 DPI	laboratori/lab04_visualizzazione/
Modulo 5	2h	Automazione della Pipeline ETL	Batch scanner glob , filtro file lock ~\$, storage Parquet compresso	laboratori/lab05_automazione_pipeline/
Modulo 6	4h	Dashboard Streamlit Interattiva	Layout reattivo, caching @st.cache_data , metric cards, simulatore What-If	laboratori/lab06_dashboard_streamlit/
Modulo 7	1h	Deploy Linux & Systemd (Live Demo)	Configurazione servizio demone, Restart=always , monitoraggio log journalctl	laboratori/lab07_deploy_linux/
Project Work	3h	Integrazione Autonoma Filiale Napoli	Bonifica dataset Napoli, re-ingestione ETL, aggiornamento Streamlit	project_work/
TOTALE	22h	Percorso Formativo Completo	Dall'Excel grezzo al servizio Linux in produzione	8 Moduli + Project Work

4. REGIA DIDATTICA COMPLETA MODULO PER MODULO

CANOVACCIO DOCENTE & GUIDA DI REGIA D'AULA

Corso: Laboratorio Python + Analisi Dati (22 Ore)

Docente: Arnaldo Morena • ITIS Campobasso • Anno 2026

Finalità del Canovaccio

Questo documento rappresenta il manuale di regia per il docente. Per ogni sessione didattica definisce:

- L'aggancio iniziale (**Apertura**) per catturare l'attenzione dell'aula.
- La narrazione aziendale (**Storytelling**) calata sulla catena *TechStore Italia*.
- Le domande strategiche (**Domande da fare all'aula**) per stimolare il ragionamento critico e testare i prerequisiti.
- La scaletta della dimostrazione a schermo (**Live Demo**).
- I punti critici e le trappole cognitive degli studenti (**Errori tipici**).
- La scansione temporale minuto per minuto (**Gestione tempi**).
- Il ponte logico verso il tema successivo (**Transizione**).

INTRODUZIONE & PATTO D'AULA (Durata: 30 min)

1. Apertura

"Benvenuti a tutti. In queste 22 ore non faremo accademia né teoria astratta: trasformeremo il vostro computer in una vera postazione di Data Engineering e Business Intelligence. Da oggi voi siete il team di analisti dati assunto da TechStore Italia, una catena retail con filiali a Roma, Milano, Torino e una nuova acquisizione a Napoli. Il vostro compito è salvare l'azienda dal caos dei fogli Excel manuali, automatizzare i flussi e costruire una dashboard che il Direttore Generale consulterà ogni mattina."

2. Storytelling

Presentare il problema reale del management:

- I direttori di filiale inviano ogni fine mese file Excel compilati a mano via email.
- I dati arrivano con duplicati, formati di data disallineati, prezzi mancanti o scritti con simboli di valuta e virgole.
- Il consolidamento manuale richiede 4 giorni lavorativi a persona ed è pieno di errori contabili.
- Obiettivo: creare una pipeline Python automatica a zero-touch che pulisce, unifica e pubblica i dati sul web.

3. Domande da fare all'aula

- **Domanda Aperta:** "Chi di voi ha mai dovuto unire a mano 5 o 6 fogli Excel diversi per fare un report? Cosa succede se il mese dopo vi chiedono di rifarlo?"
- **Domanda di Verifica:** "Perché non possiamo fidarci a fare analisi direttamente sui file inviati dalle filiali senza un processo di validazione?"

4. Demo: Ispezione del Repository e Setup

- Mostrare l'albero delle directory nel terminale e su VS Code.
- Eseguire in diretta la creazione dell'ambiente virtuale: `python3 -m venv .venv` e l'attivazione.
- Installare le dipendenze: `pip install -r requirements.txt`.
- Lanciare Jupyter Lab: `jupyter lab`.

5. Errori tipici

- Tentativo di eseguire script senza aver attivato il virtualenv (mancanza librerie).
- Confusioni su percorsi relativi e directory di lavoro (`pwd / os.getcwd()`).

6. Gestione Tempo

- **00:00 - 00:10:** Benvenuto, patto d'aula, presentazione del docente e obiettivi.
- **00:10 - 00:20:** Storytelling del caso TechStore Italia e architettura del corso.
- **00:20 - 00:30:** Verifica setup ambiente su tutte le macchine degli studenti.

7. Transizione

"Ora che la nostra officina è pronta e abbiamo gli attrezzi installati, partiamo dal motore di base: come manipolare strutture dati complesse con il Python nativo prima ancora di delegare il lavoro a librerie esterne."

MODULO 1 - PYTHON OPERATIVO PER L'ANALISI DATI (2 Ore)

Notebook di riferimento: `laboratori/lab01_python_operativo/ Soluzione: soluzioni_docente/sol01_python_operativo_commentato.py`

1. Apertura

"Prima di correre con Pandas, dobbiamo capire come Python gestisce le transazioni in memoria. Immaginate di ricevere uno stream di dati grezzi da un gestionale ERP sotto forma di lista di dizionari: dobbiamo calcolare sconti, IVA e aggregati usando solo la logica algoritmica pura."

2. Storytelling

La filiale di Roma ha inviato un estratto vendite in formato JSON/Dizionario. Il responsabile acquisti ha inserito nomi clienti con spazi superflui e caratteri minuscoli/maiuscoli incoerenti (es. `' tech solutions srl '`). Dobbiamo normalizzare le ragioni sociali e calcolare il fatturato netto e le imposte per ciascun record.

3. Domande da fare all'aula

- **Domanda Aperta:** "Se ho una lista di dizionari che rappresentano transazioni, come posso accumulare il fatturato totale per categoria merceologica senza conoscere a priori tutte le categorie presenti?"
- **Domanda di Verifica:** "Qual è il rischio di scrivere `totale = d['sconto']` invece di `totale = d.get('sconto', 0)` quando elaboriamo dati da fonti esterne?"
- **Domanda di Logica:** "Cosa accade se in un ciclo `for x in lista:` modificate o eliminate elementi direttamente da `lista`?"

4. Live Demo del Docente

1. Definire la funzione `calcola_totale_riga(quantita, prezzo, sconto, iva)`.
2. Mostrare il dizionario di accumulo `aggregato = {}` e il pattern `.get()` per raggruppare per categoria.
3. Mostrare la normalizzazione di stringhe con `.strip()`, `.split()`, `.join()` e il mapping per acronimi societari (`SrL`, `SpA`).
4. Illustrare la list comprehension filtrata: `[r for r in ordine if r['categoria'] == 'Hardware']`.

5. Errori tipici degli studenti

- **KeyError** nell'accesso a campi opzionali.
- Errori di arrotondamento e float precision (es. `0.1 + 0.2 != 0.3`): spiegare l'uso di `round(valore, 2)`.
- Confusione tra mutabilità di liste/dizionari e copie superficiali (`shallow copy` vs `deep copy`).

6. Gestione Tempo

- **00:00 - 00:25:** Tipi base, collezioni native (`List[Dict]`), funzioni pure e unpacking.

- **00:25 - 00:50:** String manipulation, list/dict comprehension, logica di accumulo.
- **00:50 - 01:40:** Esercitazione pratica studenti su **lab_01_python_operativo.ipynb**.
- **01:40 - 02:00:** Debriefing collettivo alla lavagna e confronto soluzioni.

7. Transizione

"Scrivere cicli for e dizionari a mano funziona bene per 10 righe. Ma quando i record diventano 50.000 o 1 milione, il codice puro diventa lento e complicato. È qui che entra in gioco l'arma principale del data analyst: Pandas."

MODULO 2 - PANDAS FONDAMENTALE (4 Ore)

Notebook di riferimento: **laboratori/lab02_pandas_fondamenti/ Soluzione: soluzioni_docente/sol02_pandas_fondamenti_commentato.py**

1. Apertura

"In questo modulo abbandoniamo le tabelle Excel come le avete sempre intese ed entriamo nella mentalità vettoriale. In Pandas non ragioniamo riga per riga: applichiamo operazioni istantanee su intere colonne (Series) e matrici bidimensionali (DataFrame)."

2. Storytelling

Apriamo per la prima volta il file ufficiale **dataset/raw/roma.xlsx**. Dobbiamo verificare quante vendite sono state generate dal canale E-commerce, quanti ordini superano i 1.500€ di imponibile e isolare la Top 10 dei contratti commerciali più remunerativi dell'anno.

3. Domande da fare all'aula

- **Domanda Aperta:** "Secondo voi qual è la differenza tra l'indice posizionale (il numero di riga fisico) e l'etichetta dell'indice in Pandas?"
- **Domanda di Verifica:** "Perché `df[df['Quantita'] > 5 & df['Prezzo'] > 100]` solleva un `TypeError` in Python?" (Risposta: precedenza operatori bitwise; servono le parentesi tonde).
- **Domanda Cruciale:** "Che differenza c'è tra `.loc` e `.iloc`? Quando è obbligatorio usare uno rispetto all'altro?"

4. Live Demo del Docente

1. `pd.read_excel('dataset/raw/roma.xlsx', sheet_name='Dati_Vendite')` con ispezione di `.shape`, `.info()`, `.describe()`.
2. Dimostrare la differenza tra estrazione colonna Series `df['Fatturato']` e sotto-DataFrame `df[['Fatturato', 'Quantita']]`.
3. Slicing con `.iloc[10:20, 0:4]` vs `.loc[0:10, ['Ragione_Sociale', 'Fatturato_Lordo']]`.
4. Filtri booleani composti con `&`, `|`, `~` e parentesi obbligatorie.
5. Mostrare il famigerato **SettingWithCopyWarning** provocato da un'assegnazione su vista, e risolvere in diretta applicando `.copy()`.
6. Calcolo colonne vettorializzate e ordinamento decrescente con `.sort_values(by='Fatturato_Netto', ascending=False)`.

5. Errori tipici degli studenti

- Dimenticanza del parametro **sheet_name** con caricamento del foglio errato.
- Omissione delle parentesi tonde nei filtri composti `((cond1) & (cond2))`.
- Confusione tra **ascending=True** (crescente) e **ascending=False** (decrescente).
- Manipolazioni di subset senza `.copy()` con comparsa di warning.

6. Gestione Tempo (Sessione da 4 ore, intervallata da pausa)

- **00:00 - 00:50:** Architettura Pandas, Series, DataFrame, importazione Excel, ispezione metadati.

- **00:50 - 01:40:** Selezione ed indexing: `.loc`, `.iloc`, slicing, colonne singole e multiple.
- **01:40 - 02:00:** *Pausa caffè (20 min).*
- **02:00 - 02:45:** Filtri booleani composti, problema View vs Copy, il metodo `.copy()`.
- **02:45 - 03:35:** Laboratorio pratico su **lab_02_pandas_fondamenti.ipynb** (Esercizi 2.1 - 2.4).
- **03:35 - 04:00:** Risoluzione guidata, analisi Top 10 Deals e Q&A.

7. Transizione

"Abbiamo visto come filtrare e ordinare dati corretti. Ma nella vita reale i dati non arrivano puliti. Nel prossimo modulo indosseremo i guanti da chirurgo del dato e impareremo a bonificare record sporchi, date illeggibili e a collegare tabelle diverse con il merge relazionale."

MODULO 3 - DATA WRANGLING & QUALITÀ DEL DATO (3 Ore)

Notebook di riferimento: **laboratori/lab03_data_wrangling/ Soluzione: soluzioni_docente/sol03_data_wrangling_commentato.py**

1. Apertura

"L'80% del tempo di un Data Scientist in azienda non è speso a creare modelli, ma a fare 'Data Wrangling': pulire, convertire, gestire valori nulli e unire tabelle eterogenee. Oggi imparerete a trasformare dati grezzi pieni di anomalie in un dataset certificato per il business."

2. Storytelling

Il file della filiale di Roma contiene 100 record duplicati esatti inseriti da un doppio export contabile. Inoltre molte date sono stringhe '15-Mar-2024' o numeri seriali di Excel (**45367**), e alcuni prezzi unitari mancano completamente. Dobbiamo ripulire il dataset e arricchirlo incrociando l'anagrafica clienti con un merge relazionale per scoprire quali settori industriali generano più ricavi.

3. Domande da fare all'aula

- **Domanda Aperta:** "Se abbiamo prezzi mancanti (NaN), perché in ambito business conviene imputarli con la mediana anziché con la media aritmetica?"
- **Domanda di Verifica:** "Cosa accade se fate un `pd.merge(how='left')` e la tabella a destra ha chiavi duplicate?" (Spiegare l'esplosione cartesiana delle righe).
- **Domanda Tecnica:** "A cosa serve il parametro `validate='many_to_one'` nel merge?"

4. Live Demo del Docente

1. Rilevare duplicati con `df.duplicated().sum()` e bonificare con `df.drop_duplicates().copy()`.
2. Analisi `df.isna().sum()` ed imputazione sconti nulli a zero.
3. Imputazione prezzi/quantità con `transform('mean')` o mediana globale.
4. Live coding della funzione `parse_data_flessibile(val)`: gestione seriali Excel, mapping mesi in italiano (`gen, feb, mar`), fallback `pd.to_datetime`.
5. Estrazione feature temporali: `.dt.year`, `.dt.month`, `.dt.strftime('%B')`, `.dt.to_period('Q')`.
6. Esecuzione del `pd.merge(df_clean, df_anagrafica, on='Codice_Cliente', how='left', validate='m:1')` con verifica delle righe pre/post.
7. Aggregazione avanzata con `.groupby('Settore').agg(...)`.

5. Errori tipici degli studenti

- Applicazione di `pd.to_datetime` senza specificare `dayfirst=True` con scambio involontario tra giorno e mese.
- Corruzione dei tipi colonna (es. colonna numeri che resta di tipo `object` a causa di stringhe '€' o spazi).
- Dimenticanza del `reset_index()` dopo una `groupby` multi-colonna.

6. Gestione Tempo

- **00:00 - 00:45:** Duplicati, missing values, imputazione statistica, string cleaning su codici cliente.
- **00:45 - 01:30:** Normalizzazione date disomogenee e feature engineering temporale.
- **01:30 - 02:30:** Merge relazionale (1:1, 1:N, N:1), validazione integrità e groupby settoriale (Laboratorio 3).
- **02:30 - 03:00:** Revisione collettiva del codice e certificazione output.

7. Transizione

"Ora che abbiamo un dataset pulito, coerente e arricchito con le informazioni anagrafiche, dobbiamo presentarlo al management. I manager non leggono tabelle di 5.000 righe: vogliono grafici chiari, trend evidenti e una dashboard esecutiva ad alto impatto."

MODULO 4 - VISUALIZZAZIONE & REPORTING ESECUTIVO (3 Ore)

Notebook di riferimento: **laboratori/lab04_visualizzazione/ Soluzione:**
soluzioni_docente/sol04_visualizzazione_commentato.py

1. Apertura

"Un'analisi dati brillante non serve a nulla se non riuscite a comunicarla efficacemente ai decision maker. In questo modulo abbandoniamo i grafici standard di default e impariamo a comporre un report visuale executive a 4 pannelli con qualità da pubblicazione (300 DPI)."

2. Storytelling

Il Consiglio di Amministrazione di TechStore Italia richiede un report trimestrale urgente:

1. Chi sono i migliori 8 clienti corporate?
2. Come sta andando il trend del fatturato mese per mese rispetto all'obiettivo medio?
3. Quali categorie di prodotto vendono di più nei vari canali commerciali (Direct, E-commerce, Retail)?
4. Qual è la politica di sconti applicata dalla forza vendita per canale?

3. Domande da fare all'aula

- **Domanda Aperta:** "Perché per confrontare 8 clienti con nomi lunghi è preferibile un grafico a barre orizzontali (**barh**) anziché verticali?"
- **Domanda di Verifica:** "Qual è la differenza fondamentale tra la sintassi procedurale **plt.plot()** e quella Object-Oriented **fig, ax = plt.subplots()**?"
- **Domanda Visual:** "Cosa rappresenta la linea centrale (mediana) e i bordi della scatola in un Boxplot Seaborn?"

4. Live Demo del Docente

1. Configurazione globale: **plt.style.use('seaborn-v0_8-whitegrid')** e font sizing.
2. Grafico a barre orizzontali per i Top Clienti con aggiunta di etichette dati sui vertici delle barre (**ax.text**).
3. Grafico a linee con linea di riferimento della media aziendale (**ax.axhline**) ed etichette dei mesi in italiano.
4. Heatmap di correlazione e matrice canale vs categoria con annotazioni numeriche (**sns.heatmap(..., annot=True, fmt='.1f')**).
5. Costruzione della griglia 2x2: **fig, axes = plt.subplots(2, 2, figsize=(16, 10))**.
6. Salvataggio ed esportazione ad alta definizione: **fig.savefig('executive_report.png', dpi=300, bbox_inches='tight')**.

5. Errori tipici degli studenti

- Sovrapposizione delle etichette sull'asse X (risolvere con **plt.xticks(rotation=45)** o **barh**).
- Dimenticanza di **plt.tight_layout()** con titoli o assi tagliati nel file PNG esportato.
- Dimenticanza di **plt.close()** nei cicli di generazione grafici che causa saturazione di memoria.

6. Gestione Tempo

- **00:00 - 00:45:** Architettura Object-Oriented Matplotlib (**fig**, **ax**), Bar chart orizzontali, etichette custom.
- **00:45 - 01:30:** Seaborn, lineplot con bande e medie, boxplot distribuzioni sconti, heatmap matrici.
- **01:30 - 02:30:** Laboratorio pratico su **lab_04_visualizzazione.ipynb** e composizione dashboard 2x2.
- **02:30 - 03:00:** Analisi dei report generati a 300 DPI e best practice di data storytelling.

7. Transizione

"Finora abbiamo lavorato su un singolo file (Roma) all'interno di Jupyter. Ma in azienda i dati arrivano continuamente da Roma, Milano, Torino e altre città. Nel prossimo modulo faremo il salto da Data Analyst a Data Engineer: creeremo uno script che scansiona le cartelle, elabora tutti i file in batch e genera un database ottimizzato in formato Parquet."

MODULO 5 - AUTOMAZIONE DELLA PIPELINE ETL (2 Ore)

Notebook di riferimento: **laboratori/lab05_automazione_pipeline/ Soluzione: soluzioni_docente/sol05_automazione_pipeline_commentato.py**

1. Apertura

"Oggi automatizziamo il nostro lavoro. Se un processo richiede più di due click manuali, va scritto in codice. Creeremo una pipeline ETL (Extract, Transform, Load) completa, solida, parametrizzabile da riga di comando e dotata di logging professionale."

2. Storytelling

TechStore Italia riceve mensilmente file Excel da tutte le filiali nella cartella **dataset/raw/**. Spesso i dipendenti aprono i file lasciando file temporanei di blocco (**~\$roma.xlsx**). La nostra pipeline deve scansionare la cartella, ignorare i file temporanei, processare tutti i dati, unificare l'anagrafica clienti, salvare un file master compresso in Parquet e generare un report Excel a più schede per la direzione amministrativa.

3. Domande da fare all'aula

- **Domanda Aperta:** "Cosa succede a una pipeline se un utente dell'amministrazione ha aperto un file Excel in cartella mentre lo script viene eseguito?"
- **Domanda di Verifica:** "Perché il formato Parquet è fino all'80% più compatto e 20 volte più veloce da leggere rispetto a un file Excel o CSV?"
- **Domanda Architetture:** "A cosa serve il modulo standard **logging** rispetto a semplici **print()** in uno script batch di produzione?"

4. Live Demo del Docente

1. Utilizzo di **glob.glob('dataset/raw/*.xlsx')** con filtro list comprehension: **[f for f in files if not os.path.basename(f).startswith('~\$')]**.
2. Struttura modulare a 3 funzioni: **carica_dati()**, **trasforma_dataset()**, **esporta_dataset()**.
3. Configurazione del modulo **logging** con timestamp e severity levels (**INFO**, **WARNING**, **ERROR**).
4. Esportazione colonnare in Parquet: **df.to_parquet('vendite_consolidate_italia.parquet', index=False)**.
5. Scrittura multi-foglio Excel con context manager: **with pd.ExcelWriter(...) as writer: ...**.
6. Aggiunta del supporto CLI con **argparse (--input-dir, --output-dir)**.

5. Errori tipici degli studenti

- Crash dello script causato dall'apertura accidentale di file lock **~\$*.xlsx**.
- Omissione del blocco **with** con mancata scrittura fisica del file Excel (**pd.ExcelWriter**).
- Concatenazione di DataFrame con colonne disallineate che generano centinaia di NaN.

6. Gestione Tempo

- **00:00 - 00:35:** Teoria ETL, pattern batch con glob, filtri lock file, modularità architetturale.
- **00:35 - 01:05:** Storage Parquet (Snappy), logging di produzione, ExcelWriter multi-scheda.
- **01:05 - 01:45:** Esercitazione autonoma su **lab_05_automazione_pipeline.ipynb**.
- **01:45 - 02:00:** Esecuzione da terminale dello script con argomenti CLI e verifica file generati.

7. Transizione

"Ora abbiamo una base dati pulita, unificata e velocissima in formato Parquet. Ma i dirigenti aziendali non usano il terminale né leggono script Python. Nel prossimo modulo costruiremo un'applicazione web interattiva completa con Streamlit, accessibile dal browser."

MODULO 6 - DASHBOARD STREAMLIT INTERATTIVA (4 Ore)

Notebook di riferimento: **laboratori/lab06_dashboard_streamlit/** Codice sorgente: **dashboard/app.py**

Soluzione: **soluzioni_docente/sol06_dashboard_streamlit_commentato.py**

1. Apertura

"Benvenuti nel mondo dello sviluppo web per data scientist. Con Streamlit non serve conoscere HTML, CSS o JavaScript: scriveremo un'applicazione interattiva completa, dotata di filtri reattivi, KPI cards, schede multi-tab e un simulatore economico What-If, interamente in Python puro."

2. Storytelling

La Direzione Generale vuole uno strumento live per monitorare le vendite nazionali. Il Direttore Commerciale desidera poter filtrare per filiale, categoria e periodo di tempo, e soprattutto vuole simulare in diretta cosa succederebbe al fatturato se venissero tagliati gli sconti del 5% o aumentati i volumi del 10%.

3. Domande da fare all'aula

- **Domanda Aperta:** "Come fa Streamlit ad aggiornare la pagina quando spostate uno slider o selezionate una voce da un menu a tendina?" (Risposta: riesegue l'intero script top-to-bottom).
- **Domanda Cruciale:** "Se lo script viene rieseguito a ogni interazione, come evitiamo di ricaricare il file da disco ogni volta?" (Risposta: caching con **@st.cache_data**).
- **Domanda di UI:** "Qual è il vantaggio di usare **st.tabs()** per suddividere l'analisi rispetto a una pagina verticale infinita?"

4. Live Demo del Docente

1. Avvio dell'app da terminale: **streamlit run dashboard/app.py**.
2. Spiegazione di **st.set_page_config(layout='wide')** e iniezione CSS per personalizzare le metric cards.
3. Dimostrazione del decoratore **@st.cache_data(ttl=600)** per caricare il Parquet una sola volta in RAM.
4. Costruzione della Sidebar laterale (**st.sidebar.multiselect**, **st.sidebar.date_input**).
5. Rendering delle KPI cards in 5 colonne (**st.columns(5)** e **st.metric**).
6. Creazione della navigazione a schede (**st.tabs**):
 - Tab 1: Trend e grafici integrati con **st.pyplot(fig)**.
 - Tab 2: Classifica clienti con tabelle formattate (**st.dataframe**).
 - Tab 3: Simulatore What-If con slider interattivi e calcolo delta in tempo reale.
 - Tab 4: Tabella dati interattiva con barra di ricerca e pulsante di download CSV (**st.download_button**).

5. Errori tipici degli studenti

- Dimenticanza di **@st.cache_data** con app lenta e poco reattiva.
- Modifica in-place del DataFrame caricato in cache (che corrompe i dati per gli altri utenti): spiegare la necessità di **.copy()**.

- Chiamate di `st.set_page_config()` a metà script anziché come prima istruzione Streamlit.

6. Gestione Tempo (Sessione da 4 ore con pausa)

- **00:00 - 00:50:** Architettura reattiva, ciclo di vita Streamlit, caching @`st.cache_data`, setup pagina.
- **00:50 - 01:40:** Sidebar, widget interattivi, filtri dinamici, KPI cards a colonne.
- **01:40 - 02:00:** *Pausa (20 min).*
- **02:00 - 02:45:** Implementazione Multi-tab, grafici Matplotlib/Seaborn in Streamlit, What-If simulator.
- **02:45 - 03:40:** Esercitazione guidata su `lab_06_dashboard_streamlit` e sviluppo `app.py`.
- **03:40 - 04:00:** Test completo in locale, esportazione dati CSV e Q&A.

7. Transizione

"La nostra dashboard funziona perfettamente sul computer locale. Ma cosa succede se chiudiamo il laptop? L'applicazione smette di funzionare e nessuno in azienda può più vederla. Nel prossimo modulo vedremo come distribuire la nostra app su un server Linux di produzione, trasformandola in un servizio demone sempre attivo."

MODULO 7 - DEPLOY LINUX & SYSTEMD (1 Ora - Live Demo Guidata)

Guida di riferimento: `laboratori/lab07_deploy_linux/` Script di riferimento:

`soluzioni/sol07_deploy_linux.sh` Soluzione: `soluzioni_docente/sol07_deploy_linux_commentato.md`

1. Apertura

"Completiamo il ciclo del software portando il nostro progetto in produzione. Oggi vedremo cosa significa fare il deploy su un server Linux Cloud/VPS: configureremo un demone Systemd che avvierà automaticamente la nostra dashboard all'avvio del sistema e la riavvierà in 5 secondi in caso di crash."

2. Storytelling

Il reparto IT di TechStore Italia ha assegnato un'istanza virtuale Linux Ubuntu Server su cui ospitare la piattaforma. Dobbiamo garantire che il servizio risponda 24 ore su 24 sulla porta di rete 8501, sia resiliente ai riavvii di macchina e permetta all'amministratore di monitorare i log applicativi in tempo reale.

3. Domande da fare all'aula

- **Domanda Aperta:** "Se lanciamo `streamlit run app.py` da un terminale SSH e poi chiudiamo la finestra, perché l'applicazione si spegne?" (Segnale `SIGHUP` inviato alla chiusura della sessione).
- **Domanda di Verifica:** "A cosa serve la direttiva `Restart=always` e `RestartSec=5` in un unit file systemd?"
- **Domanda di Sicurezza:** "Perché non dovremmo MAI eseguire un'applicazione web come utente `root`?"

4. Live Demo del Docente (Modalità Live Demo Guidata)

1. Connessione SSH simulata/reale all'ambiente Linux.
2. Ispezione della struttura del file `/etc/systemd/system/dashboard_vendite.service`.
3. Analisi riga per riga delle sezioni `[Unit]`, `[Service]`, `[Install]`.
4. Spiegazione dei percorsi assoluti per l'eseguibile Python/Streamlit dell'ambiente virtuale.
5. Esecuzione dei comandi di controllo di sistema:
 - `sudo systemctl daemon-reload`
 - `sudo systemctl enable --now dashboard_vendite.service`
 - `sudo systemctl status dashboard_vendite.service`
6. Ispezione in streaming continuo dei log applicativi con `sudo journalctl -u dashboard_vendite.service -f`.
7. Simulazione di un crash (`kill -9`) per mostrare il recupero e riavvio automatico in 5 secondi da parte di Systemd.
8. Cenni architetturali sull'uso di Nginx come Reverse Proxy su porta 80/443 con certificati HTTPS.

5. Errori tipici degli studenti

- Inserimento di percorsi relativi (es. `./venv/bin/streamlit` invece di percorsi assoluti completi).
- Dimenticanza di `sudo systemctl daemon-reload` dopo la modifica del file di servizio.
- Mancanza di permessi di lettura/esecuzione sulla cartella del progetto per l'utente designato.

6. Gestione Tempo (60 min tassativi)

- **00:00 - 00:15:** Architettura Client-Server, limiti di esecuzione da sessione interattiva, concetto di demone.
- **00:15 - 00:30:** Anatomia dell'Unit File systemd, direttive di servizio, isolamento privilegi.
- **00:30 - 00:50:** Live demo guidata: installazione, gestione ciclo di vita (`systemctl`), streaming log (`journalctl`), test di resilienza al crash.
- **00:50 - 01:00:** Discussione architetturale, cenni Nginx e transizione al Project Work.

7. Transizione

"Ora avete tutte le competenze teoriche e pratiche necessarie: Python puro, Pandas, Data Wrangling, Visualizzazione, Automazione ETL, Streamlit e Deploy Linux. È il momento di mettervi alla prova da soli con il Project Work finale."

PROJECT WORK FINALE - INTEGRAZIONE FILIALE NAPOLI (3 Ore)

Documentazione di riferimento: `project_work/` Benchmark: `project_work/traccia_studenti.md` e `project_work/criteri_valutazione.md`

1. Apertura e Consegna della Traccia

"Questo è il momento della certificazione sul campo. Non siete più studenti guidati dal docente, ma consulenti data analyst autonomi. L'azienda TechStore Italia ha appena completato l'acquisizione della filiale di Napoli e vi ha consegnato il file grezzo `dataset/raw/napoli_project_work.xlsx`. Avete 3 ore per eseguire l'audit dei dati, aggiornare la pipeline ETL, rigenerare il master Parquet a 4 filiali, verificare la dashboard Streamlit e produrre il report esecutivo finale."

2. Storytelling

La filiale di Napoli invia dati non conformi: 100 record duplicati esatti, date formattate con stili misti, missing values nei prezzi e codici cliente non normalizzati. Il successo dell'integrazione dipende dalla capacità degli studenti di applicare in autonomia la catena di trasformazione già collaudata per le altre filiali.

3. I 4 Deliverable da Richiedere agli Studenti

1. **Deliverable 1:** Notebook o script di pulizia dedicato per la filiale di Napoli con verifica benchmark (€ 1.042.850,50 fatturato netto).
2. **Deliverable 2:** Esecuzione della pipeline ETL su tutte e 4 le filiali e generazione di `vendite_consolidate_italia.parquet` (esattamente 4.552 record).
3. **Deliverable 3:** Verifica e screenshot della dashboard Streamlit con inclusione della filiale di Napoli e KPI aggiornati.
4. **Deliverable 4:** Generazione del report Excel direzionale consolidato a 4 fogli e salvataggio della dashboard grafica 2x2.

4. Benchmark Numerici Ufficiali per il Docente (Foglio di Controllo)

- **Dataset Napoli Grezzo:** 1.100 righe
- **Dataset Napoli Pulito:** 1.000 record (100 duplicati eliminati)
- **Fatturato Netto Napoli:** € 1.042.850,50
- **Master Dataset Italia (4 Filiali):** 4.552 record totali
- **Fatturato Netto Consolidato Nazionale:** € 4.614.820,50
- **Margine Lordo Totale:** € 1.712.440,20

5. Rubrica di Valutazione Docente (100 Punti)

- **25 pt - Data Wrangling & Qualità:** Deduplicazione, date normalizzate, imputazione corretta, assenza NaN residui.
- **25 pt - Ingegneria ETL:** Scansione batch, filtro lock file, merge relazionale con anagrafica, export Parquet.
- **25 pt - Dashboard Streamlit:** Reattività, filtri multi-filiale coerenti, simulatore What-If funzionante.
- **25 pt - Reporting & Best Practice:** Report Excel multi-foglio, export a 300 DPI, codice pulito e documentato.
- *Soglia certificazione: 60/100 • Eccellenza con lode didattica: >= 90/100.*

6. Gestione Tempo (180 min totali)

- **00:00 - 00:20:** Presentazione traccia di lavoro, requisiti di consegna e chiarimento dubbi.
 - **00:20 - 02:20:** Sviluppo autonomo da parte degli studenti (il docente assume il ruolo di Project Manager e risponde solo a quesiti architetturali).
 - **02:20 - 02:45:** Raccolta deliverable e verifica automatica contro i benchmark ufficiali.
 - **02:45 - 03:00:** Chiusura del corso, feedback complessivo, consegna attestati e conclusioni.
-
-

5. CRONOPROGRAMMA MINUTO PER MINUTO (1.320 MINUTI)

PIANO DI GESTIONE DEI TEMPI & SCANSIONE ORARIA (22 ORE)

Corso: Laboratorio Python + Analisi Dati

Docente: Arnaldo Morena • ITIS Campobasso

Panoramica della Distribuzione Didattica

Il corso ha una durata complessiva certificata di **22 Ore** (1.320 minuti netti). La metodologia adottata è un modello **Hands-On** (**40% Spiegazione / Live Demo - 60% Laboratorio Attivo**).

+-----+ DISTRIBUZIONE 22 ORE +-----+			
Modulo	Ore	Teoria/Demo	Laboratorio
+-----+			
Modulo 1 - Python Operat.	2h	45 min	75 min
Modulo 2 - Pandas Fondam.	4h	90 min	150 min
Modulo 3 - Data Wrangling	3h	65 min	115 min
Modulo 4 - Visualizzaz.	3h	65 min	115 min
Modulo 5 - Automazione	2h	40 min	80 min
Modulo 6 - Streamlit	4h	85 min	155 min
Modulo 7 - Deploy Linux	1h	45 min	15 min
Project Work Finale	3h	20 min	160 min
+-----+			
TOTALE	22h	~ 7.5h	~ 14.5h
+-----+			

CRONOPROGRAMMA DETTAGLIATO MODULO PER MODULO

MODULO 1: Python Operativo (2 Ore • 120 min)

Minuti	Attività Didattica	Tipologia	Obiettivo Operativo
00 - 15	Accoglienza, patto d'aula, introduzione TechStore Italia	Lezione	Allineamento aspettative e obiettivi
15 - 35	Tipi nativi, collezioni List[Dict] , funzioni e calcolo KPI	Live Demo	Implementazione calcola_totale_riga()
35 - 50	String manipulation, acronimi societari, list comprehension	Live Demo	Pulizia ragioni sociali e filtri
50 - 95	Laboratorio 1: Esercizi 1.1, 1.2, 1.3, 1.4	Hands-On	Autonomia su accumulo dizionari
95 - 110	Correzione collettiva e debriefing errori comuni	Revisione	Fissaggio concetti e best practice
110 - 120	Buffer & Q&A: Consolidamento e ponte a Pandas	Buffer	Risoluzione dubbi e transizione

MODULO 2: Pandas Fondamentale (4 Ore • 240 min)

Minuti	Attività Didattica	Tipologia	Obiettivo Operativo	
00 - 40	Architettura Series vs DataFrame, importazione da Excel	Lezione/Demo	<code>pd.read_excel(), .shape, .info()</code>	
40 - 75	Indicizzazione ed estrazione: <code>.loc</code> vs <code>.iloc</code> , slicing	Live Demo	Selezione mirata di righe e colonne	
75 - 115	Laboratorio 2 (Parte 1): Esercizi 2.1 e 2.2	Hands-On	Ispezione e selezioni posizionali	
115 - 130	<i>Pausa Caffè Rigenerativa</i>	Pausa	Recupero energie	
130 - 165	Filtri booleani composti (<code>&</code> , <code>~</code>)	), View vs Copy e .copy()	Live Demo	Eliminare SettingWithCopyWarning
165 - 205	Laboratorio 2 (Parte 2): Esercizi 2.3 e 2.4 (Top Deals)	Hands-On	Colonne calcolate e ordinamenti	
205 - 225	Discussione Top 10 Deals ed estrazione insight	Revisione	Interpretazione di business del dato	
225 - 240	Buffer & Q&A: Verifica setup su tutte le postazioni	Buffer	Recupero studenti in ritardo	

MODULO 3: Data Wrangling & Qualità del Dato (3 Ore • 180 min)

Minuti	Attività Didattica	Tipologia	Obiettivo Operativo
00 - 35	Deduplicazione, analisi missing values, imputazione con mediana	Live Demo	<code>drop_duplicates(), fillna()</code>
35 - 65	Parsing date disomogenee (<code>parse_data_flessibile</code>) e feature tempo	Live Demo	Gestione seriali Excel e stringhe italiane
65 - 100	Laboratorio 3 (Parte 1): Pulizia, date e imputazione prezzi	Hands-On	Bonifica completa dataset Roma
100 - 130	Merge relazionale con anagrafica, validazione <code>many_to_one</code>	Live Demo	Prevenire esplosione cartesiana
130 - 165	Laboratorio 3 (Parte 2): Merge e GroupBy settoriale	Hands-On	Aggregazioni aggregate con <code>.agg()</code>
165 - 180	Buffer & Validazione: Verifica coerenza record	Buffer	Controllo asserzioni numeriche

MODULO 4: Visualizzazione & Reporting Esecutivo (3 Ore • 180 min)

Minuti	Attività Didattica	Tipologia	Obiettivo Operativo
00 - 35	Architettura OOP Matplotlib (<code>fig, ax</code>), Barh per Top Clienti	Live Demo	Etichette sui vertici, layout pulito
35 - 65	Seaborn: lineplot con medie, boxplot sconti, heatmap matrici	Live Demo	Grafici statistici avanzati
65 - 105	Laboratorio 4 (Parte 1): Grafici 4.1, 4.2 e 4.3	Hands-On	Generazione singoli grafici su file
105 - 135	Composizione dell'Executive Dashboard 2x2 ed export 300 DPI	Live Demo	Layout a 4 pannelli coordinati
135 - 165	Laboratorio 4 (Parte 2): Assemblaggio dashboard 2x2	Hands-On	Finalizzazione file <code>executive_report.png</code>
165 - 180	Buffer & Feedback: Revisione grafica dei report	Buffer	Allineamento estetico e leggibilità

MODULO 5: Automazione della Pipeline ETL (2 Ore • 120 min)

Minuti	Attività Didattica	Tipologia	Obiettivo Operativo
00 - 25	Pattern di scansione batch con glob , filtro lock file (~\$)	Live Demo	Ingestione multi-file automatizzata
25 - 45	Storage Parquet compresso (Snappy) vs Excel, logging di produzione	Live Demo	.to_parquet() , modulo logging
45 - 90	Laboratorio 5: Scrittura ed esecuzione script ETL completo	Hands-On	Generazione master Parquet e report Excel
90 - 110	Test da riga di comando con argomenti CLI (argparse)	Live Demo/Lab	Esecuzione parametrica da terminale
110 - 120	Buffer: Controllo dimensioni file e tempi di lettura	Buffer	Verifica performance Parquet

MODULO 6: Dashboard Interattiva Streamlit (4 Ore • 240 min)

Minuti	Attività Didattica	Tipologia	Obiettivo Operativo
00 - 45	Modello reattivo Streamlit, ciclo di vita, caching @ st.cache_data	Live Demo	Setup pagina wide e lettura cached
45 - 80	Sidebar, selettori multi-filiale, filtri data, KPI cards in colonne	Live Demo	st.sidebar , st.columns , st.metric
80 - 120	Laboratorio 6 (Parte 1): Costruzione struttura e filtri	Hands-On	Interfaccia base reattiva funzionante
120 - 135	<i>Pausa Rigenerativa</i>	Pausa	Recupero concentrazione
135 - 165	Navigazione Multi-Tab, grafici dinamici, What-If simulator, export	Live Demo	st.tabs , simulatore di scenario, download
165 - 215	Laboratorio 6 (Parte 2): Implementazione What-If ed export	Hands-On	Completamento applicazione app.py
215 - 240	Buffer & Demo: Presentazione incrociata delle dashboard	Buffer	Test utente interattivo

MODULO 7: Deploy Linux & Systemd (1 Ora • 60 min)

Modalità: Live Demo Guidata dal Docente

Minuti	Attività Didattica	Tipologia	Obiettivo Operativo
00 - 15	Architettura Cloud/Server, limiti sessioni SSH, concetto di demone	Lezione	Comprensione del ciclo di vita in produzione
15 - 30	Creazione dell'unit file /etc/systemd/system/dashboard_vendite.service	Live Demo	Sintassi Unit File, policy Restart=always
30 - 45	Attivazione servizio (systemctl enable --now) e streaming log (journalctl -f)	Live Demo	Controllo operativo demone in tempo reale
45 - 55	Test di resilienza: uccisione forzata del processo e riavvio automatico	Live Demo	Dimostrazione della robustezza Systemd
55 - 60	Cenni su Nginx Reverse Proxy (porte 80/443 SSL) e Q&A	Discussione	Quadro architetturale enterprise completo

PROJECT WORK FINALE: Filiale Napoli (3 Ore • 180 min)

Minuti	Attività Didattica	Tipologia	Obiettivo Operativo
00 - 20	Consegna della traccia, spiegazione dei 4 deliverable e benchmark	Presentazione	Comprensione dei requisiti aziendali
20 - 75	Fase 1: Audit e pulizia dati filiale Napoli (1.000 record netti)	Lavoro Autonomo	Bonifica anomalie e verifica € 1.042.850,50
75 - 125	Fase 2: Esecuzione ETL a 4 filiali, generazione Parquet (4.552 righe)	Lavoro Autonomo	Consolidamento master e report Excel
125 - 155	Fase 3: Aggiornamento dashboard Streamlit e verifica grafici	Lavoro Autonomo	Test interattivo con filtro Napoli attivo
155 - 170	Valutazione e confronto con benchmark del docente (100 pt)	Valutazione	Certificazione competenze acquisite
170 - 180	Conclusioni finali del corso, feedback e rilascio attestati	Chiusura	Termine del percorso formativo

STRATEGIE DI RECUPERO RITARDI & ACCELERATORI D'AULA

Scenario A: L'aula è in ritardo sulla tabella di marcia (-30 min)

1. **Modulo 2:** Ridurre l'esercitazione sui filtri concentrandosi solo sull'Esercizio 2.3 e 2.4 (i più qualificanti).
2. **Modulo 3:** Fornire la funzione `parse_data_flessibile()` pre-scritta nel notebook invece di farla digitare da zero, concentrando l'attenzione sul merge relazionale.
3. **Modulo 4:** Limitare la dashboard 2x2 a 2 grafici essenziali (Trend + Boxplot) rimandando i restanti alla consultazione della soluzione.

Scenario B: L'aula è particolarmente avanzata (+30 min di anticipo)

1. **Approfondimento Modulo 3:** Introdurre l'imputazione contestuale per categoria tramite `df.groupby('Categoria')['Prezzo'].transform('median')`.
2. **Approfondimento Modulo 4:** Mostrare come salvare il grafico in formato vettoriale PDF/SVG per stampe ad altissima fedeltà.
3. **Approfondimento Modulo 6:** Aggiungere un grafico a barre interattivo Plotly in Streamlit (`st.plotly_chart`).
4. **Approfondimento Modulo 7:** Mostrare la configurazione di un virtual host di base in Nginx (`/etc/nginx/sites-available/streamlit`).

6. CHECKLIST OPERATIVA D'AULA

Checklist Pre-Corso (Setup Iniziale - T-60 min)

- [] Verificare che l'interprete Python 3.10+ sia correttamente installato su tutte le postazioni.
- [] Verificare la presenza del virtual environment **.venv** e l'installazione di tutti i pacchetti da **requirements.txt**.
- [] Verificare che la cartella **dataset/raw/** contenga i 4 file Excel integri (**roma.xlsx**, **milano.xlsx**, **torino.xlsx**, **napoli_project_work.xlsx**).
- [] Testare l'avvio del server Jupyter Notebook o Jupyter Lab.
- [] Testare il comando **streamlit hello** o **streamlit run dashboard/app.py** sulla porta 8501.
- [] Proiettare la slide 1 (Titolo e benvenuto) sul videoproiettore principale.

Checklist Pre-Modulo (Routine per ciascun Modulo)

- [] Proiettare la slide introduttiva del modulo corrispondente con gli obiettivi orari.
- [] Aprire il notebook starter per gli studenti in **laboratori/** e la soluzione docente in **soluzioni_docente/**.
- [] Lanciare la domanda di Hook iniziale (da Canovaccio) prima di scrivere codice.
- [] Impostare il timer visivo per l'esercitazione pratica degli studenti.

Checklist Pre-Project Work (Modulo 8 - T-15 min)

- [] Verificare che tutti gli studenti abbiano generato con successo il file **vendite_consolidate_italia.parquet** (3 filiali).
 - [] Distribuire la traccia **project_work/traccia_studenti.md**.
 - [] Proiettare la tabella dei criteri di valutazione a 100 punti (**project_work/criteri_valutazione.md**).
 - [] Chiarire il benchmark ufficiale di Napoli (€ 1.042.850,50 fatturato netto) e del consolidato nazionale (€ 4.614.820,50).
-

7. PIANO DI EMERGENZA & DISASTER RECOVERY D'AULA

TROUBLESHOOTING D'AULA: GUIDA ALLA RISOLUZIONE DEI PROBLEMI

Corso: Laboratorio Python + Analisi Dati (22 Ore)

Docente: Arnaldo Morena • ITIS Campobasso

Questa guida è uno strumento di pronto intervento rapido per il docente e per gli assistenti d'aula. Per ogni problema frequente viene descritto il **Sintomo riscontrato**, la **Causa radice** e la **Soluzione immediata (Quick Fix)**.

1. PROBLEMI PYTHON, AMBIENTI VIRTUALI & DEPENDENCIES

1.1 ModuleNotFoundError: No module named 'pandas' (o 'streamlit', 'seaborn')

- **Sintomo:** Lo script o il notebook fallisce subito alla prima riga di **import**.
- **Causa Radice:** L'ambiente virtuale **.venv** non è stato attivato nel terminale, oppure Jupyter Lab sta eseguendo un kernel Python globale di sistema invece del kernel virtualenv.
- **Quick Fix:**
 1. Da terminale: **source .venv/bin/activate** (Linux/Mac) o **.venv\Scripts\activate** (Windows).
 2. In Jupyter Lab: verificare in alto a destra che il kernel selezionato sia **.venv (Python 3.x)**. Se non compare:

```
.venv/bin/python -m ipykernel install --user --name=corso_python --display-name="Python (Corso ITIS)"
```

1.2 SyntaxError: invalid syntax su operatore di tipo o f-string

- **Sintomo:** Python segnala errore di sintassi su **list[dict]** o su formattazioni con f-string.
- **Causa Radice:** Lo studente sta usando una versione obsoleta di Python (es. Python 3.7 o 3.8).
- **Quick Fix:**

Verificare la versione: **python3 --version**. Se inferiore a 3.10, installare Python 3.10+ o usare la sintassi compatibile **typing.List** e **typing.Dict**.

1.3 PermissionError: [Errno 13] Permission denied su file Excel

- **Sintomo:** Impossibile leggere o sovrascrivere un file **.xlsx** durante l'esecuzione di uno script.
- **Causa Radice:** Il file Excel è aperto in un'altra finestra con Microsoft Excel o LibreOffice Calc, che ha posto un lock esclusivo sul file.
- **Quick Fix:**

Chiudere l'applicazione Excel/Calc e rieseguire lo script.

2. PROBLEMI PANDAS & DATA WRANGLING

2.1 SettingWithCopyWarning: A value is trying to be set on a copy of a slice from a DataFrame

- **Sintomo:** Compare un rettangolo rosso con warning durante l'assegnazione di una colonna (`df_sub['nuova_col'] = ...`).
- **Causa Radice:** `df_sub` è stato creato tramite un filtro senza aver invocato il metodo `.copy()`, lasciando Pandas nell'incertezza tra copia e vista.
- **Quick Fix:**

Aggiungere esplicitamente `.copy()` al momento del filtraggio:

```
# CORRETTO:
df_sub = df.loc[df['Quantita'] > 5].copy()
df_sub['Fatturato_Netto'] = df_sub['Quantita'] * df_sub['Prezzo_Unitario']
```

2.2 MergeError: Merge keys have non-unique values in right frame; cannot validate many-to-one

- **Sintomo:** Il merge relazionale si blocca con eccezione **MergeError**.
- **Causa Radice:** La tabella anagrafica contiene chiavi duplicate (es. due clienti con lo stesso **Codice_Cliente**), violando la regola di integrità `validate='many_to_one'`.
- **Quick Fix:**

Deduplicare la tabella di destra sulla chiave primaria prima del merge:

```
df_anagrafica_clean = df_anagrafica.drop_duplicates(subset=['Codice_Cliente']).copy()
df_merged = pd.merge(df_clean, df_anagrafica_clean, on='Codice_Cliente', how='left', validate='m:1')
```

2.3 Date invertite o non riconosciute (giorno e mese scambiati)

- **Sintomo:** Una data come **05/06/2024** (5 Giugno) viene parsata come **2024-05-06** (6 Maggio).
- **Causa Radice:** `pd.to_datetime()` di default utilizza il formato anglosassone mese-prima (`monthfirst=True`).
- **Quick Fix:**

Specificare `dayfirst=True` o definire il formato esplicito:

```
df['Data_dt'] = pd.to_datetime(df['Data_Vendita'], format='mixed', dayfirst=True)
```

2.4 Numeri letti come stringhe (tipo object) con € o virgole

- **Sintomo:** Le operazioni matematiche falliscono con **TypeError: can't multiply sequence by non-int of type 'float'**.
- **Causa Radice:** I prezzi importati contengono caratteri non numerici (spazi, simboli di valuta €, virgole per i decimali).
- **Quick Fix:**

Applicare una pipeline di normalizzazione testuale prima della conversione numerica:

```
df['Prezzo_Pulito'] = (
    df['Prezzo_Unitario'].astype(str)
    .str.replace('€', '', regex=False)
    .str.replace(',', '.', regex=False)
    .str.strip()
)
df['Prezzo_Pulito'] = pd.to_numeric(df['Prezzo_Pulito'], errors='coerce')
```

3. PROBLEMI VISUALIZZAZIONE (MATPLOTLIB & SEABORN)

3.1 Etichette dell'asse X sovrapposte e illeggibili

- **Sintomo:** Le ragioni sociali dei clienti o i nomi dei mesi si sovrappongono formando una striscia nera illeggibile.
- **Causa Radice:** Mancanza di rotazione o troppi elementi sull'asse orizzontale.
- **Quick Fix:**
 1. Ruotare le etichette: `ax.tick_params(axis='x', rotation=45)` o `plt.xticks(rotation=45)`.
 2. Per molti elementi con testi lunghi, passare a un grafico a barre orizzontali: `ax.barh(...)`.

3.2 Grafico tagliato ai bordi nel file PNG salvato su disco

- **Sintomo:** Il titolo superiore o i valori dell'asse Y non compaiono nell'immagine esportata.
- **Causa Radice:** I margini della figura non sono stati ricalcolati automaticamente prima del salvataggio.
- **Quick Fix:**

Invocare `plt.tight_layout()` e specificare `bbox_inches='tight'` in `savefig`:

```
plt.tight_layout()
fig.savefig('report.png', dpi=300, bbox_inches='tight')
```

3.3 Warning UserWarning: The figure layout has changed to tight

- **Sintomo:** Compare un avviso durante l'esecuzione di più chiamate a `tight_layout()`.
- **Causa Radice:** Chiamata ridondante di `tight_layout()` dopo che il motore grafico ha già finalizzato la geometria.
- **Quick Fix:**

Chiamare `plt.tight_layout()` una sola volta subito prima di `plt.show()` o `fig.savefig()`.

4. PROBLEMI STREAMLIT & WEB DASHBOARD

4.1 StreamlitAPIException: set_page_config() can only be called once per app page, and must be the first Streamlit command used

- **Sintomo:** L'applicazione Streamlit si blocca con schermata di errore rossa.
- **Causa Radice:** È stato inserito un comando come `st.title()` o `st.write()` prima di `st.set_page_config()`.
- **Quick Fix:**

Spostare `st.set_page_config(...)` in cima allo script come prima istruzione Streamlit in assoluto.

4.2 L'applicazione web è lenta e si ricarica faticosamente a ogni click

- **Sintomo:** Ogni volta che l'utente muove uno slider o seleziona un filtro, compare il logo 'Running' per diversi secondi.
- **Causa Radice:** La funzione di caricamento del dataset (`load_dataset()`) o le elaborazioni pesanti non sono decorate con `@st.cache_data`.
- **Quick Fix:**

Aggiungere il decoratore di caching:

```
@st.cache_data(ttl=600)
def load_dataset():
    return pd.read_parquet('dataset/generated/vendite_consolidate_italia.parquet')
```

4.3 OSError: [Errno 98] Address already in use (Porta 8501 occupata)

- **Sintomo:** Il comando `streamlit run app.py` fallisce indicando che la porta di rete è già utilizzata.
- **Causa Radice:** Un'altra istanza di Streamlit è già in esecuzione in background sulla porta di default 8501.
- **Quick Fix:**

1. Specificare una porta alternativa: `streamlit run app.py --server.port 8502`.
2. Oppure terminare il processo precedente:

```
fuser -k 8501/tcp # Su Linux
killall streamlit # Termina tutti i processi Streamlit attivi
```

4.4 I grafici Matplotlib mostrano warning o layout rimpiccioliti nella dashboard

- **Sintomo:** Compare `PylotGlobalUseWarning` nella console.
- **Causa Radice:** Utilizzo di `st.pyplot()` senza passare esplicitamente l'oggetto `fig`.
- **Quick Fix:**

Creare sempre esplicitamente la figura e passarla al widget:

```
fig, ax = plt.subplots(figsize=(10, 5))
# ... codice di plotting ...
st.pyplot(fig)
plt.close(fig)
```

5. PROBLEMI LINUX, SSH & SYSTEMD DEPLOY

5.1 Failed to start service: Unit ... not found

- **Sintomo:** Il comando `sudo systemctl start dashboard_vendite.service` fallisce.
- **Causa Radice:** Il file `.service` non è stato copiato in `/etc/systemd/system/` oppure non è stato eseguito `daemon-reload`.
- **Quick Fix:**

```
sudo cp /tmp/dashboard_vendite.service /etc/systemd/system/  
sudo systemctl daemon-reload  
sudo systemctl start dashboard_vendite.service
```

5.2 Lo stato del servizio è failed (Result: exit-code)

- **Sintomo:** `sudo systemctl status dashboard_vendite.service` mostra lo stato in rosso con errore di terminazione.
- **Causa Radice:** Percorso errato dell'interprete virtualenv in **ExecStart** o permessi non concessi all'utente specificato in **User=**.
- **Quick Fix:**

Ispezionare i log dettagliati dell'errore:

```
sudo journalctl -u dashboard_vendite.service -e --no-pager
```

Verificare che il percorso in **ExecStart** sia assoluto e che il file Python esista realmente in quel percorso.

5.3 L'applicazione funziona localmente ma non è raggiungibile dall'esterno (browser degli altri PC)

- **Sintomo:** Il browser restituisce *"Impossibile raggiungere il sito"* collegandosi all'IP del server (`http://<IP_SERVER>:8501`).
- **Causa Radice:**
 1. Streamlit è stato avviato in ascolto solo su **localhost** (127.0.0.1) anziché su tutte le interfacce di rete (**0.0.0.0**).
 2. Il firewall di Linux (**ufw**) blocca la porta TCP 8501.
- **Quick Fix:**
 1. Assicurarsi che nel comando o file di configurazione sia presente **--server.address 0.0.0.0**.
 2. Aprire la porta nel firewall:

```
sudo ufw allow 8501/tcp  
sudo ufw reload
```

8. FAQ D'AULA RIORGANIZZATE PER MODULO (53+ DOMANDE)

FAQ AULA: 50+ DOMANDE FREQUENTI CON RISPOSTE DOCENTE

Corso: Laboratorio Python + Analisi Dati (22 Ore)

Docente: Arnaldo Morena • ITIS Campobasso

Questo archivio raccoglie oltre 50 domande reali poste dagli studenti durante lo svolgimento del corso, suddivise per area tematica, con le relative risposte didattiche e spiegazioni operative per il docente.

SEZIONE 1: PYTHON OPERATIVO & FONDAMENTI

1. Perché in Python usiamo una `List[Dict]` invece di matrici o array per rappresentare dati tabellari all'inizio?

Risposta Docente: Una lista di dizionari rispecchia fedelmente la struttura di un recordset relazionale o di un payload JSON proveniente da un'API REST o da un database NoSQL. Ogni elemento della lista è una riga (record), e le chiavi del dizionario rappresentano i nomi delle colonne. È il modo più intuitivo e flessibile per comprendere il concetto di tabella prima di passare a Pandas.

2. Qual è la differenza tra `d['chiave']` e `d.get('chiave', default)`?

Risposta Docente: L'accesso diretto `d['chiave']` genera un'eccezione irreversibile **KeyError** se la chiave non esiste nel dizionario, interrompendo lo script. Il metodo `d.get('chiave', default)` restituisce invece il valore di fallback specificato (es. `0` o `None`) senza sollevare errori, rendendo il codice robusto nella gestione di dati incompleti.

3. Perché non posso modificare una lista mentre la sto iterando in un ciclo `for`?

Risposta Docente: Modificare la dimensione di una lista durante l'iterazione sballa i puntatori interni degli indici, provocando il salto di elementi o comportamenti indefiniti. La best practice consiste nell'iterare su una copia superficiale `for x in list(mia_lista):` oppure nell'utilizzare una list comprehension per creare una nuova lista filtrata.

4. Che differenza c'è tra `==` e `is` in Python?

Risposta Docente: L'operatore `==` confronta l'uguaglianza dei valori contenuti negli oggetti (`val1 == val2`), mentre `is` confronta l'identità in memoria, ovvero verifica se due variabili puntano esattamente alla medesima locazione fisica di RAM (`id(a) == id(b)`).

5. Perché scriviamo `imponibile * (1 - sconto / 100.0)` con `.0` sul denominatore?

Risposta Docente: Sebbene in Python 3 la divisione tra interi produca un float (`/`), l'uso esplicito del letterale float (`100.0`) rende chiaro al lettore del codice che l'operazione avviene nel dominio continuo dei numeri a virgola mobile e preserva la compatibilità semantica.

6. Come funziona l'unpacking delle tuple in un ciclo su dizionario?

Risposta Docente: Il metodo `d.items()` restituisce un iteratore di tuple composte da (`chiave`, `valore`). Scrivendo `for k, v in d.items():`, Python assegna automaticamente il primo elemento della tupla a `k` e il secondo a `v`.

7. Perché i calcoli con i numeri decimali a volte danno risultati strani come `0.30000000000000004`?

Risposta Docente: Dipende dallo standard IEEE 754 per la rappresentazione in virgola mobile binaria: molte frazioni decimali (come 0.1 o 0.2) sono numeri periodici in base 2. In ambito contabile/commerciale si risolve applicando la funzione `round(valore, 2)` o utilizzando il modulo standard `decimal.Decimal`.

SEZIONE 2: PANDAS FONDAMENTALE

8. Perché Pandas è molto più veloce dei cicli for in Python?

Risposta Docente: Pandas poggia su array NumPy scritti in C e C++. Quando eseguiamo operazioni tra colonne (es. `df['Qta'] * df['Prezzo']`), il calcolo avviene a livello di codice compilato a basso livello tramite operazioni vettoriali (SIMD), senza l'overhead dell'interprete Python per ogni singolo elemento.

9. Qual è la differenza strutturale tra una Series e un DataFrame?

Risposta Docente: Una **Series** è un array monodimensionale etichettato con un solo indice. Un **DataFrame** è una tabella bidimensionale formata da più **Series** affiancate che condividono lo stesso indice di riga (**index**).

10. Quando devo usare `.loc[]` e quando `.iloc[]`?

Risposta Docente: Usiamo `.loc[]` quando selezioniamo righe e colonne in base alle loro **etichette** (nomi di colonna ed etichette di indice). Usiamo `.iloc[]` quando vogliamo effettuare una selezione puramente **posizionale** tramite numeri interi da 0 a **N-1**.

11. Perché nel filtro `df[(df['A'] > 10) & (df['B'] < 5)]` servono obbligatoriamente le parentesi tonde?

Risposta Docente: In Python gli operatori logici bitwise (`&`, `|`, `~`) hanno una precedenza sintattica superiore rispetto agli operatori di confronto (`>`, `<`, `==`). Senza parentesi, Python cercherebbe di calcolare `10 & df['B']`, generando un **TypeError**.

12. Che cos'è esattamente il `SettingWithCopyWarning` e perché si verifica?

Risposta Docente: È un avviso che Pandas emette quando tentiamo di modificare un sottoinsieme di dati estratto con un filtro. Pandas non sa con certezza se quel sottoinsieme sia una **Copia** indipendente in memoria o una **Vista** collegata al DataFrame originale. Per eliminarlo ed evitare corruzioni di memoria, è necessario chiamare esplicitamente `.copy()` al momento del filtraggio.

13. Come fa `pd.read_excel()` a leggere i file senza avere Microsoft Excel installato sulla macchina?

Risposta Docente: Pandas utilizza engine open source come **openpyxl** o **calamine**, che analizzano direttamente il formato OpenXML (che internamente è un archivio compresso di file XML) senza dipendere dal software Excel proprietario.

14. Perché `df.shape` non ha le parentesi tonde alla fine come `df.head()`?

Risposta Docente: Perché **shape** è un **attributo** (una tupla memorizzata nell'oggetto che contiene le dimensioni correnti (**righe**, **colonne**)), mentre **head()** è un **metodo** (una funzione che accetta argomenti ed esegue un'elaborazione prima di restituire le prime N righe).

15. A cosa serve `reset_index(drop=True)`?

Risposta Docente: Quando filtriamo o ordiniamo un DataFrame, i vecchi numeri di riga rimangono invariati (es. righe 3, 15, 84). `reset_index(drop=True)` rigenera un indice sequenziale continuo da 0 a **N-1**, eliminando il vecchio indice per evitare che diventi una colonna separata.

SEZIONE 3: DATA WRANGLING & QUALITÀ DATI

16. Perché si consiglia di fare `drop_duplicates(subset=[...])` invece che un `drop_duplicates()` generico?

Risposta Docente: `drop_duplicates()` generico elimina solo le righe dove **tutte** le colonne sono identiche al 100%. Spesso due record duplicati differiscono per un timestamp di pochi secondi o una nota testuale. Specificare `subset=['ID_Transazione']` o `subset=['Data', 'Cliente', 'Prodotto']` assicura l'eliminazione della duplicazione logica del business.

17. Qual è la differenza tra `df.isna().sum()` e `df.isnull().sum()`?

Risposta Docente: In Pandas `isna()` e `isnull()` sono esattamente la stessa identica funzione: `isnull()` è un alias storico mantenuto per analogia con il linguaggio SQL e con la libreria R.

18. Quando è opportuno eliminare le righe con valori nulli (`dropna()`) e quando è preferibile imputarli (`fillna()`)?

Risposta Docente: Si eliminano le righe con `dropna()` solo quando la percentuale di dati mancanti è trascurabile (< 1-2%) oppure quando il campo mancante è la chiave primaria irrecuperabile (es. `ID_Transazione`). Si imputano i valori con `fillna()` (usando mediana, media o zero) quando la colonna è un attributo quantitativo secondario (es. sconti o prezzi) per non perdere il volume complessivo delle altre transazioni.

19. Perché le date di Excel lette come numeri interi (es. 45367) corrispondono a date reali?

Risposta Docente: Microsoft Excel memorizza internamente le date come il numero di giorni trascorsi a partire dal 1° gennaio 1900 (con un noto bug storico che considera il 1900 bisestile). Per convertirle in Python basta sommare quei giorni alla data base 1899-12-30.

20. Cosa significa `errors='coerce'` in `pd.to_datetime()`?

Risposta Docente: Indica a Pandas che, qualora incontri una stringa di data non valida, corrotta o indecifrabile (es. `'Data_Sconosciuta'`), non deve lanciare un'eccezione che blocca lo script, ma deve convertire quel valore nel valore speciale `pd.NaT` (Not a Time), consentendo al resto della pipeline di proseguire.

21. Qual è la differenza tra una `join inner`, `left`, `right` e `outer`?

Risposta Docente:

- **inner:** mantiene solo i record con corrispondenza in entrambe le tabelle.
- **left:** mantiene tutti i record della tabella sinistra e inserisce `NaN` dove non c'è corrispondenza a destra.
- **right:** mantiene tutti i record della tabella destra.
- **outer:** mantiene tutti i record di entrambe le tabelle, riempiendo con `NaN` le mancanze.

22. Che cos'è l'esplosione cartesiana in un `pd.merge()` e come si previene?

Risposta Docente: Si verifica quando la tabella di destra (es. anagrafica) contiene chiavi duplicate per lo stesso codice cliente. In questo caso, ogni riga della tabella sinistra viene moltiplicata tante volte quante sono le occorrenze a destra, alterando falsamente il totale delle vendite. Si previene specificando `validate='many_to_one'` nel merge o deduplicando preventivamente la tabella anagrafica.

23. A cosa serve il parametro `indicator=True` in `pd.merge()`?

Risposta Docente: Aggiunge una colonna speciale `_merge` con valori `'both'`, `'left_only'`, `'right_only'`, permettendo all'analista di verificare istantaneamente quali record non hanno trovato corrispondenza con la tabella anagrafica.

SEZIONE 4: VISUALIZZAZIONE & REPORTING GRAFICO

24. Perché Matplotlib Object-Oriented (`fig`, `ax`) è superiore alla sintassi `plt.*`?

Risposta Docente: La sintassi `plt.*` fa affidamento su un puntatore globale nascosto alla figura "corrente". Quando si creano layout complessi a griglia (es. 2x2), la sintassi OOP permette di controllare esplicitamente e simultaneamente ogni singolo asse (`ax[0, 0]`, `ax[0, 1]`, ecc.) senza rischio di sovrascritture accidentali.

25. Come si fa a salvare un grafico con risoluzione adatta alla stampa tipografica o a monitor 4K?

Risposta Docente: Si imposta il parametro `dpi=300` (Dots Per Inch) e `bbox_inches='tight'` nel metodo `fig.savefig('report.png', dpi=300, bbox_inches='tight')`.

26. Qual è il significato statistico del Boxplot Seaborn?

Risposta Docente: La linea spessa centrale è la **mediana** (50° percentile), i bordi della scatola sono il **primo quartile Q1** (25°) e il **terzo quartile Q3** (75°). La distanza tra Q3 e Q1 è l'intervallo interquartile (IQR). I "baffi" estendono i dati fino a $1.5 * IQR$, mentre i singoli punti oltre i baffi sono considerati **outlier** statistici.

27. Perché nei grafici a barre Seaborn a volte vediamo una barretta nera verticale sopra le barre?

Risposta Docente: È la barra di errore (intervallo di confidenza al 95%). Se vogliamo mostrare la somma totale del fatturato senza intervalli statistici, impostiamo `estimator='sum'` ed `errorbar=None`.

28. Perché dopo aver generato e salvato un grafico dobbiamo sempre chiamare `plt.close(fig)`?

Risposta Docente: Matplotlib conserva ogni figura creata nella memoria RAM finché non viene esplicitamente chiusa. In script automatizzati o pipeline che generano decine di grafici, omettere `plt.close()` provoca un memory leak progressivo e rallentamenti di sistema.

29. Come si invertono gli assi in un grafico a barre orizzontali in Matplotlib?

Risposta Docente: Si utilizza `ax.barh(y, width)` invece di `ax.bar(x, height)`. Per avere il cliente con il valore più alto in cima, si ordina preventivamente la serie in senso crescente (`ascending=True`).

30. Come formattare le etichette dell'asse Y per visualizzare migliaia di Euro (€ 150k)?

Risposta Docente: Dividendo i valori per `1000.0` prima del plot e impostando `ax.set_ylabel('Fatturato (k€)')`, oppure utilizzando un formattatore Matplotlib: `ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, loc: f"€ {x*1e-3: .0f}k"))`.

SEZIONE 5: AUTOMAZIONE PIPELINE & STORAGE

31. Perché in una pipeline batch usiamo `glob.glob()` con un filtro per `~$`?

Risposta Docente: Quando un utente apre un file Excel, il sistema operativo crea nella stessa cartella un file temporaneo di lock nascosto che inizia con `~$` (es. `~$roma.xlsx`). Se lo script tenta di leggerlo con Pandas solleva un **PermissionError** o un errore di formato corrotto.

32. Perché il formato Parquet è preferibile a CSV ed Excel per i dati consolidati?

Risposta Docente: Parquet è un formato colonnare binario compresso (algoritmo Snappy). Preserva i tipi di dato nativi (le date rimangono date, i numeri rimangono float32/int64), occupa fino all'80% di spazio in meno su disco e consente letture molto più veloci perché legge solo le colonne richieste dalla query.

33. Come fa `pd.ExcelWriter` a creare file con più fogli di lavoro?

Risposta Docente: Invocando `pd.ExcelWriter(percorso, engine='openpyxl')` all'interno di un blocco `with`, l'oggetto `writer` mantiene aperto l'archivio Excel e consente chiamate successive a `df.to_excel(writer, sheet_name='NomeFoglio')` prima di finalizzare la scrittura alla chiusura del blocco.

34. A cosa serve il modulo standard `logging` al posto dei normali `print()`?

Risposta Docente: `logging` aggiunge automaticamente timestamp, livello di gravità (**INFO**, **WARNING**, **ERROR**), nome del modulo, e consente di reindirizzare i messaggi simultaneamente su console e su file di log permanenti (`pipeline.log`) senza dover modificare il codice sorgente.

35. Perché è buona norma incapsulare il codice batch all'interno di `if __name__ == '__main__':`?

Risposta Docente: Perché consente di eseguire lo script direttamente da riga di comando come programma principale, ma permette anche di importare le sue singole funzioni (es. `trasforma_dataset()`) all'interno di altri script (come la dashboard Streamlit o i test di regressione) senza eseguire l'intero ciclo batch.

36. Come gestire i parametri CLI con `argparse` in uno script ETL?

Risposta Docente: Definendo un parser:

```
python
parser = argparse.ArgumentParser()
parser.add_argument('--input-dir', default='dataset/raw')
args = parser.parse_args()
`
```

Questo permette di richiamare lo script da cron job o script bash con percorsi personalizzati:
`python script.py --input-dir /dati/nuovi`.

SEZIONE 6: DASHBOARD STREAMLIT

37. Qual è il paradigma di esecuzione reattivo di Streamlit?

Risposta Docente: Ogni volta che l'utente interagisce con un widget (muove uno slider, cambia un filtro), Streamlit non esegue solo un handler di evento locale, ma riesegue l'intero script Python dall'inizio alla fine (top-to-bottom), rigenerando l'interfaccia con i nuovi parametri.

38. Come funziona il decoratore `@st.cache_data` e quando si invalida la cache?

Risposta Docente: `@st.cache_data` calcola l'hash dei parametri di input della funzione. Se la funzione viene richiamata con gli stessi argomenti già visti, Streamlit salta l'esecuzione del corpo della funzione e restituisce direttamente il risultato memorizzato in RAM. La cache si invalida se cambiano i parametri di input, se cambia il codice della funzione o allo scadere del parametro `ttl` (Time To Live).

39. Perché `st.set_page_config()` deve essere la prima istruzione Streamlit dello script?

Risposta Docente: Perché Streamlit invia le direttive di layout (titolo della scheda del browser, favicon, layout wide) nell'header iniziale della sessione WebSocket; qualsiasi comando visivo eseguito prima impedisce la corretta inizializzazione del viewport.

40. Come si dispongono i widget o le KPI cards su più colonne in Streamlit?

Risposta Docente: Si usa `col1, col2, col3 = st.columns(3)` e poi si scrive all'interno delle singole colonne tramite blocco `with col1`: oppure chiamando direttamente `col1.metric('Titolo', 'Valore')`.

41. Come funziona il componente `st.download_button` per esportare file CSV?

Risposta Docente: Converte il DataFrame filtrato in una stringa di byte UTF-8 tramite `df.to_csv(index=False).encode('utf-8')` e fornisce al browser un pulsante standard HTML5 di download che non richiede roundtrip di salvataggio su disco sul server.

42. Come si realizza un simulatore economico What-If in Streamlit?

Risposta Docente: Si collegano due o più slider (`st.slider`) a variabili di variazione percentuale (es. % **sconto**, % **volume**) e si applicano queste variabili a una formula vettoriale sul DataFrame filtrato, confrontando il totale ricalcolato con il totale reale tramite il parametro `delta` di `st.metric`.

43. Che differenza c'è tra `@st.cache_data` e `@st.cache_resource`?

Risposta Docente: `@st.cache_data` si usa per oggetti serializzabili e dati tabellari (DataFrame, liste, dizionari). `@st.cache_resource` si usa per oggetti non serializzabili e connessioni globali a stato persistente (connessioni a database SQLAlchemy, modelli di Machine Learning, client HTTP).

SEZIONE 7: DEPLOY LINUX & SYSTEMD

44. Perché non si deve lasciare un'applicazione Streamlit in esecuzione in un terminale SSH con `nohup` o `screen` in produzione?

Risposta Docente: `nohup` o `screen` non offrono gestione dei crash, non riavviano l'applicazione al boot del server, non gestiscono i log in modo centralizzato con rotazione automatica e non permettono il governo tramite strumenti standard di orchestrazione DevOps. Systemd è lo standard de facto su Linux per tutti i servizi enterprise.

45. Cosa significa la sezione `[Unit]` in un file `.service` di Systemd?

Risposta Docente: Contiene i metadati descrittivi del servizio (**Description**) e le dipendenze di avvio (`After=network.target`), indicando a Systemd di non avviare il processo finché lo stack di rete del sistema operativo non è completamente operativo.

46. Perché dobbiamo specificare il percorso assoluto sia per l'eseguibile Python sia per lo script in `ExecStart`?

Risposta Docente: Systemd viene eseguito in un ambiente di esecuzione minimale che non eredita il **PATH** o le variabili di sessione dell'utente interattivo. Specificare percorsi assoluti (es. `/home/ubuntu/app/.venv/bin/streamlit`) garantisce l'uso del corretto interprete virtualenv.

47. Qual è la differenza tra `systemctl enable` e `systemctl start`?

Risposta Docente: `systemctl enable` registra il servizio nei target di avvio del sistema per farlo partire automaticamente al boot della macchina. `systemctl start` avvia immediatamente il processo nel momento presente. Il comando combinato `systemctl enable --now` esegue entrambe le operazioni contemporaneamente.

48. Come si controllano i log applicativi di un servizio Systemd in tempo reale?

Risposta Docente: Eseguendo `sudo journalctl -u nome_servizio.service -f`. Il flag `-f` (follow) mantiene il terminale agganciato allo stream di log, stampando ogni nuova riga non appena viene generata.

49. A cosa serve un Reverse Proxy Nginx davanti a Streamlit?

Risposta Docente: Permette di esporre l'applicazione sulle porte standard **80** (HTTP) e **443** (HTTPS) con certificati crittografici SSL/TLS (Let's Encrypt), gestire il buffering delle richieste e proteggere l'applicazione Streamlit (che risponde sulla porta interna 8501) da accessi diretti non autorizzati.

50. Cosa accade se si modifica il file `.service` e si prova subito a fare `systemctl restart` senza fare `daemon-reload`?

Risposta Docente: *Systemd ignora le modifiche apportate sul file di testo e riavvia il servizio con la vecchia configurazione presente nella cache in RAM, mostrando un messaggio di avviso "Unit file changed on disk, run 'systemctl daemon-reload' to reload units".*

SEZIONE 8: PROJECT WORK & INTEGRAZIONE DATI

51. Come deve comportarsi uno studente se durante l'integrazione di Napoli il numero totale di righe non corrisponde a 4.552?

Risposta Docente: *Deve verificare i due punti di perdita o moltiplicazione dati:*

- 1. Verificare che siano stati eliminati esattamente i 100 record duplicati di Napoli (da 1.100 a 1.000 righe).*
- 2. Verificare che nel merge con l'anagrafica non ci siano state perdite di record o duplicazioni da chiavi non univoche.*

52. Cosa fare se il fatturato netto finale non coincide con il benchmark ufficiale di € 4.614.820,50?

Risposta Docente: *L'errore è quasi sempre causato dall'ordine di applicazione dello sconto (calcolare lo sconto sull'imponibile prima di sottrarlo) o da un'imputazione errata dei prezzi mancanti (es. aver imputato con la media invece che con la mediana, o non aver gestito gli sconti NaN impostandoli a zero).*

9. VALUTAZIONE E CORREZIONE DEL PROJECT WORK

Tabella Ufficiale dei Benchmark di Controllo (Foglio Risolutivo Docente)

- **Dataset Napoli Grezzo:** 1.100 righe
- **Dataset Napoli Pulito:** 1.000 record netti (100 duplicati eliminati)
- **Fatturato Netto Filiale Napoli:** € 1.042.850,50
- **Master Dataset Italia (4 Filiali):** 4.552 record totali (o 4.700 righe comprensive di record di controllo)
- **Fatturato Netto Consolidato Nazionale:** € 4.614.820,50
- **Margine Lordo Totale:** € 1.712.440,20

Rubrica di Valutazione a 100 Punti

- 1. Data Wrangling & Qualità Dati (25 Punti):**
 - Corretta rimozione dei 100 duplicati di Napoli (5 pt)
 - Parsing robusto delle date eterogenee (8 pt)
 - Imputazione corretta dei prezzi/quantità mancanti (7 pt)
 - Normalizzazione codici cliente e categorie (5 pt)
 - 2. Ingegneria Pipeline ETL (25 Punti):**
 - Scansione batch automatica di tutte le filiali con **glob** (8 pt)
 - Filtro di esclusione lock file **~\$** (5 pt)
 - Merge relazionale validato **many_to_one** con anagrafica (7 pt)
 - Salvataggio Parquet compresso (5 pt)
 - 3. Dashboard Streamlit Multi-Filiale (25 Punti):**
 - Caricamento cached con **@st.cache_data** (7 pt)
 - Inclusione dinamica di Napoli nei filtri della sidebar (8 pt)
 - Aggiornamento coerente delle KPI metric cards e dei grafici (5 pt)
 - Reattività del simulatore What-If (5 pt)
 - 4. Reporting & Best Practice (25 Punti):**
 - Generazione report Excel multi-foglio con **ExcelWriter** (10 pt)
 - Esportazione dashboard grafica 2x2 ad alta risoluzione (300 DPI) (8 pt)
 - Pulizia del codice, commenti e aderenza PEP 8 (7 pt)
-

10. CHIUSURA E DEBRIEFING DEL CORSO

Script Finale di Chiusura del Docente (Arnaldo Morena)

"Complimenti a tutti. In queste 22 ore intense avete compiuto un salto professionale straordinario. Siete partiti manipolando semplici dizionari Python e siete arrivati a progettare un'architettura completa di livello enterprise: ingestione dati automatica, bonifica delle anomalie, database colonnare Parquet, cruscotti web interattivi su Streamlit e deploy come servizio di produzione Linux.

Non siete più semplici utilizzatori di fogli di calcolo: siete Data Analyst e Pipeline Engineer capaci di governare il ciclo di vita del dato dalla sorgente alla decisione aziendale. Portate questo metodo nei vostri progetti futuri: automatizzate ogni processo ripetitivo, verificate sempre l'integrità dei dati e comunicate con chiarezza attraverso grafici ad alto impatto. Buon lavoro e ad maiora!"

11. APPENDICI TECNICHE DI CONSULTAZIONE RAPIDA

Appendice A: Cheat Sheet Python Operativo

- `d.get(k, default)`: Accesso sicuro alle chiavi di un dizionario.
- `[f(x) for x in list if cond]`: List comprehension filtrata.
- `" ".join(s.strip().split())`: Normalizzazione stringhe con spazi multipli.
- `round(val, 2)`: Arrotondamento decimale per grandezze monetarie.

Appendice B: Cheat Sheet Pandas

- `df = pd.read_excel('file.xlsx', sheet_name='Dati')`: Ingestione da foglio di calcolo.
- `df.loc[(df['A'] > 5) & (df['B'] < 10)]`: Filtro booleano composto.
- `df_sub = df.loc[filtro].copy()`: Eliminazione del **SettingWithCopyWarning**.
- `df.drop_duplicates(subset=['ID'])`: Deduplicazione logica di business.
- `df['Data'] = pd.to_datetime(df['Data'], format='mixed', dayfirst=True)`: Normalizzazione date.
- `pd.merge(a, b, on='ID', how='left', validate='many_to_one')`: Merge relazionale validato.
- `df.to_parquet('out.parquet', engine='pyarrow')`: Export colonnare compresso.

Appendice C: Cheat Sheet Streamlit

- `st.set_page_config(layout='wide')`: Configurazione layout a tutto schermo.
- `@st.cache_data(ttl=600)`: Caching in memoria dei dati pesanti.
- `st.sidebar.multiselect('Filtro', opzioni)`: Filtro interattivo laterale.
- `st.metric('KPI', 'Valore', delta='Delta')`: Scheda metrica con indicatore di trend.
- `tab1, tab2 = st.tabs(['Grafici', 'Dati'])`: Navigazione a schede orizzontali.
- `st.download_button('Scarica', data=csv_bytes, mime='text/csv')`: Download diretto dati.

Appendice D: Cheat Sheet Linux & Systemd

- `sudo systemctl daemon-reload`: Ricarica i file di servizio dopo una modifica.
- `sudo systemctl enable --now app.service`: Abilita l'avvio al boot e lancia subito il servizio.
- `sudo systemctl status app.service`: Controlla lo stato di esecuzione e l'occupazione RAM.
- `sudo journalctl -u app.service -f`: Visualizza i log applicativi in streaming continuo.
- `sudo ufw allow 8501/tcp`: Apre la porta del firewall per consentire accessi esterni.

Appendice E: Mappa Strutturale del Repository

- `dataset/raw/`: File Excel sorgente originali delle filiali regionali.
 - `dataset/generated/`: Output Parquet, report Excel direzionali e grafici PNG ad alta definizione.
 - `laboratori/`: Notebook ed esercizi guidati per ciascun modulo.
 - `soluzioni/`: Soluzioni ufficiali essenziali.
 - `soluzioni_docente/`: Soluzioni didattiche complete commentate a 4 livelli pedagogici.
 - `KIT_DOCENTE/`: Manuali di regia, FAQ aula, gestione tempi e troubleshooting.
 - `dashboard/`: Codice sorgente della web application Streamlit di produzione (**app.py**).
 - `slides/`: Deck ufficiale di 75 slide e script di generazione PPTX/PDF.
 - `dispensa/`: Manuale completo dello studente in formato DOCX e PDF.
 - `project_work/`: Traccia, benchmark e criteri di valutazione per l'esame finale.
-